

Trabajo Práctico 3

I. Bases de Datos NoSQL y Relacionales

Inciso 1

¿Cuales de los siguientes conceptos de RDBMS existen en MongoDB? En caso de no existir, ¿hay alguna alternativa? ¿Cual es?

- Base de Datos
- Tabla / Relacion
- Fila / Tupla
- Columna

Respuesta:

En MongoDB, la terminología y la forma de organizar la información difieren del modelo relacional tradicional (RDBMS), aunque existen equivalencias directas para la mayoría de los conceptos.

A continuación se detalla la existencia o alternativa para cada concepto:

- **Base de Datos:** Existe en MongoDB con el mismo nombre. En una instancia de MongoDB pueden coexistir múltiples bases de datos, cumpliendo un rol similar al concepto de "esquema" (schema) en un RDBMS como Oracle.
- **Tabla / Relación:** No existe con ese nombre; la alternativa es la **Colección (Collection)**. Al igual que una tabla agrupa filas, una colección agrupa documentos, con la diferencia de que MongoDB es "sin esquema" (*schemaless*), permitiendo que los documentos de una misma colección tengan estructuras diferentes.
- **Fila / Tupla:** No existe con ese nombre; la alternativa es el **Documento (Document)**. El documento es la unidad básica de información y suele estar representado en formatos como JSON o BSON. A diferencia de una fila, un documento puede contener estructuras jerárquicas complejas, como listas o documentos incrustados (subobjetos).
- **Columna:** No existe de forma fija o predefinida en el nivel de la colección; la alternativa son los **Campos o Atributos (Fields/Attributes)** dentro de cada documento. Dado que MongoDB no impone un esquema rígido, no es

necesario definir las "columnas" de antemano; cada documento puede tener sus propios campos y se pueden agregar nuevos atributos a un documento sin afectar a los demás registros de la colección.

Inciso 2

¿Existen claves foraneas en MongoDB? ¿Que diferencias existen con las bases de datos de tipo relacional?

Respuesta:

No existen las claves foráneas como un mecanismo de integridad referencial gestionado por el motor de la base de datos. Mientras que en una base de datos relacional (RDBMS) el sistema garantiza que un ID referenciado en otra tabla sea válido, en MongoDB esta responsabilidad suele recaer en la **lógica de la aplicación**.

A continuación, se detallan las principales diferencias entre MongoDB y las bases de datos relacionales:

1. Manejo de Relaciones

- **RDBMS (Normalización):** Utiliza claves foráneas y operaciones de **JOIN** para vincular tablas separadas, evitando la duplicación de datos.
- **MongoDB (Desnormalización):** Fomenta la **incrustación (embedding)** de documentos relacionados dentro de un único documento para que el acceso sea más rápido (ej. incluir los ítems directamente en el documento del pedido). También puede usar **referencias** (guardar el ID de otro documento), pero sin validación automática de integridad.

2. Esquema de Datos

- **RDBMS:** Posee un **esquema fijo** y rígido. Es necesario definir tablas y columnas antes de insertar datos.
- **MongoDB:** Es "**sin esquema**" (**schemaless**). Los documentos dentro de una misma colección pueden tener estructuras diferentes y se pueden agregar nuevos campos sobre la marcha sin afectar a los registros existentes.

3. Escalabilidad

- **RDBMS:** Está diseñado principalmente para **escalar verticalmente** (servidores más grandes). El funcionamiento en clústeres es complejo y no es nativo para el modelo.

- **MongoDB:** Diseñado para **escalar horizontalmente** de forma nativa mediante técnicas de **Sharding** (partición de datos en múltiples nodos) y replicación, lo que permite manejar volúmenes masivos de datos ("Big Data") de forma eficiente.

4. Transacciones y Consistencia

- **RDBMS:** Prioriza la consistencia fuerte mediante transacciones **ACID** que pueden abarcar múltiples tablas.
- **MongoDB:** Las operaciones suelen ser atómicas a nivel de un **solo documento**. En entornos distribuidos, a menudo utiliza el modelo **BASE** (consistencia eventual) para favorecer la disponibilidad y el rendimiento.

Inciso 3

Para acelerar las consultas, MongoDB tiene soporte para índices. ¿Que tipos de índices soporta?

Respuesta:

Para mejorar el rendimiento de las búsquedas, MongoDB permite la creación de diversos tipos de **índices** que se aplican sobre los campos de los documentos en una colección.

Los tipos de índices que soporta, según el material de la cátedra, son:

- **Índice de un solo campo (Single Field):** Es el tipo más básico, donde se indexa un único atributo del documento. Se puede aplicar tanto a campos de primer nivel como a campos dentro de documentos incrustados (por ejemplo, `db.records.createIndex( { "location.state": 1 } )`).
- **Índice compuesto (Compound Index):** Permite indexar **múltiples campos** en un solo índice, lo que es útil para consultas que filtran por varios criterios simultáneamente. Un ejemplo es `db.products.createIndex( { "item": 1, "stock": 1 } )`.
- **Índices geoespaciales (Geospatial Index):** Están diseñados específicamente para almacenar y consultar datos de coordenadas. Utilizan especificaciones como **2dsphere** para habilitar consultas de proximidad o de inclusión dentro de un área geográfica (usando operadores como `$geoWithin`).

- **Índice de Clave Primaria (_id):** MongoDB crea de forma automática un índice único sobre el campo `_id`, el cual funciona como la clave primaria del documento.

A diferencia de los almacenes de clave-valor puros donde solo se puede buscar por la clave principal, las bases de datos de documentos como MongoDB permiten crear estos índices basados en el **contenido interno** y la estructura jerárquica del documento (agregado).

Inciso 4

En MongoDB existen dos tipos de vistas. Explicar brevemente cuales son y que diferencias existen entre ellas. Además, mencionar algunos casos donde podría utilizarlas.

Respuestas:

1. Vistas Estándar (o Dinámicas)

Son consultas que actúan como una tabla virtual. No almacenan datos por sí mismas, sino que se definen mediante una lógica de cálculo sobre las colecciones base.

- **Funcionamiento:** Cada vez que el cliente accede a la vista, la base de datos ejecuta el cálculo y devuelve los resultados en ese momento.
- **Consistencia:** Al calcularse en el momento, la información siempre está actualizada según los datos más recientes de las colecciones base.

2. Vistas Materializadas

Son consultas cuyos resultados se han **precalculado** y almacenado físicamente en el disco como si fueran una colección normal.

- **Funcionamiento:** En lugar de procesar la lógica en cada lectura, el sistema lee directamente los resultados ya guardados.
- **Actualización:** Pueden actualizarse de forma **ávida** (al mismo tiempo que los datos base) o mediante **procesos por lotes** (batch) a intervalos regulares.

Diferencias Principales

Característica	Vista Estándar	Vista Materializada
Almacenamiento	Solo guarda la definición (la consulta).	Ocupa espacio físico en disco.
Rendimiento de Lectura	Puede ser costosa si el cálculo es complejo.	Extremadamente rápida (es una lectura directa).
Frescura de Datos	Siempre al día.	Puede tener cierto grado de obsolescencia.
Costo de Escritura	Nulo (solo afecta a la colección base).	Alto si es "ávida", ya que debe actualizar dos sitios a la vez.

Casos de Uso Recomendados

- **Utilizar Vistas Estándar cuando:**
 - Se requiere **encapsulamiento**: ocultar al cliente si los datos son base o derivados sin preocuparse por la frescura.
 - Los datos base cambian con mucha frecuencia y las lecturas no son masivas.
- **Utilizar Vistas Materializadas cuando:**
 - **Análisis y Estadísticas**: Para calcular promedios, sumas o conteos complejos (ej. el rating promedio de un producto basado en miles de reviews) mediante procesos de **map-reduce**.
 - **Alto Volumen de Lectura**: Cuando la aplicación necesita consultar datos derivados constantemente y no puede permitirse el costo de calcularlos cada vez.
 - **Reorganización de Datos**: Cuando se necesita ver la información organizada de una forma totalmente distinta a como se guardó originalmente en los agregados.

Inciso 5

Los documentos de una colección pueden diferir en la cantidad y tipos de campos. ¿Existen algunas formas de validar los elementos a insertar en una colección para evitar esta disparidad?

Respuesta:

Para evitar la disparidad de campos y tipos y validar los elementos a insertar, existen las siguientes estrategias según las fuentes:

1. Validación en la Capa de Aplicación

Dado que el esquema se traslada al código, la **responsabilidad de la integridad** de la base de datos recae principalmente en la aplicación.

- **Suposiciones del código:** Los programas asumen que ciertos nombres de campo están presentes y tienen tipos de datos específicos (por ejemplo, que "cantidad" sea un entero y no un texto).
- **Capa de traducción:** Es fundamental contar con una capa de traducción adecuada entre el dominio y la base de datos que se encargue de validar y convertir los datos correctamente.

2. Validación Nativa de la Base de Datos

A diferencia de los almacenes de clave-valor que ven los datos como una "masa de bits" opaca, una **base de datos documental** puede ver la estructura del documento.

- **Límites de estructura:** La base de datos puede imponer límites sobre lo que se puede colocar en ella, **definiendo las estructuras y los tipos permitidos**.
- **Validación de tipos:** En algunos casos, se pueden especificar tipos de contenido o clases de validación para asegurar que los datos cumplan con el formato esperado antes de ser confirmados.

3. Encapsulamiento de la Interacción

Una forma de reducir inconsistencias cuando múltiples aplicaciones acceden a los mismos datos es **encapsular toda la interacción** con la base de datos en un único servicio web. Al centralizar el acceso, una sola lógica de validación garantiza que los datos no se manipulen de forma inconsistente.

4. Técnica de Migración Incremental

Cuando el esquema de la aplicación cambia, se puede usar la **migración incremental** para evitar disparidades con datos antiguos:

- El nuevo código debe ser capaz de analizar tanto la estructura vieja como la nueva.

- Al guardar o actualizar un documento, este se **migra automáticamente** a la última versión del esquema, manteniendo la uniformidad de los datos activos con el tiempo.
-

En resumen, la validación se logra principalmente a través de la disciplina en el código de la aplicación (esquema implícito) y el uso de las capacidades de la base de datos para inspeccionar y restringir la estructura interna de los documentos.

Inciso 6

MongoDB tiene soporte para transacciones, pero no es igual que el de los RDBMS.

¿Cual es el alcance de una transaccion en MongoDB?

Respuesta:

- **Unidad Atómica (El Documento):** El alcance de la atomicidad en MongoDB se limita a un **único documento** (agregado). Esto significa que las actualizaciones son atómicas únicamente dentro de los límites de ese documento específico.
- **Consistencia Lógica:** Al tratar al documento como la unidad mínima, el sistema garantiza que la información tenga sentido de forma interna. Por ejemplo, si un pedido y sus ítems están incrustados en el mismo documento, se pueden actualizar juntos de forma segura.
- **Diferencia con RDBMS:** A diferencia de las bases de datos relacionales, que permiten manipular cualquier combinación de filas de diferentes tablas en una sola transacción ACID, las bases de datos orientadas a documentos no suelen ofrecer transacciones que abarquen **múltiples agregados** de forma nativa.
- **Compensación de Diseño:** Debido a esta limitación técnica, el diseño del modelo de datos en MongoDB es crítico; el desarrollador debe decidir qué datos deben estar juntos en un mismo agregado para asegurar que las actualizaciones necesarias se realicen de forma atómica.

En resumen, mientras que en un RDBMS el alcance transaccional es la base de datos completa (multitabla), en MongoDB el alcance estándar es el **documento individual**.

Inciso 7

Las relaciones entre documentos en MongoDB pueden establecerse mediante documentos embebidos o referencias. Investigar como se implementa cada una y analizar las ventajas y desventajas de cada una, comparandola con la forma estandar de establecer relaciones en una base de datos relacional.

Respuesta:

En **MongoDB**, el modelado de datos y la forma de establecer relaciones entre documentos dependen de cómo la aplicación consume la información. Al ser una base de datos orientada a **agregados**, se busca almacenar datos que suelen consultarse juntos en una única estructura rica.

A continuación, se detalla la implementación, ventajas y desventajas de las dos estrategias principales, comparándolas con el modelo relacional:

1. Documentos Embebidos (Incrustación)

Consiste en guardar documentos relacionados como **subobjetos o listas** dentro de un único documento principal. Es la forma natural de representar relaciones en bases de datos orientadas a documentos.

- **Implementación:** Por ejemplo, en lugar de tener una tabla de "Pedidos" y otra de "Ítems", se guarda el pedido con una matriz (array) de ítems directamente en su interior.
- **Ventajas:**
 - **Rendimiento de lectura:** Permite recuperar toda la información relacionada con una sola consulta a la base de datos, evitando operaciones costosas de unión (*joins*).
 - **Atomicidad:** Las actualizaciones son atómicas dentro de los límites de un único documento (agregado).
- **Desventajas:**
 - **Límite de tamaño:** Los documentos tienen un límite de tamaño físico; si la lista de elementos embebidos crece indefinidamente (ej. "1 a millones"), se puede alcanzar dicho límite.
 - **Duplicación de datos:** Puede generar redundancia e inconsistencias si el mismo dato embebido debe actualizarse en varios lugares.

2. Referencias

Esta estrategia es similar a la utilizada en las bases de datos relacionales. Se guarda el identificador (`_id`) de un documento en otro para vincularlos.

- **Implementación:** Se mantienen colecciones separadas (ej. Clientes y Pedidos). El documento de Pedido guarda el `customer_id`. Para obtener los datos del cliente, la aplicación lee el pedido, obtiene el ID y realiza una **segunda consulta** para traer los datos del cliente.
- **Ventajas:**
 - **Normalización:** Evita la duplicación de datos y es ideal cuando la información referenciada cambia con frecuencia o es accedida de forma independiente por otras aplicaciones.
 - **Flexibilidad:** Es mejor para relaciones complejas o cuando la cantidad de elementos relacionados es muy grande (evitando problemas de tamaño del documento).
- **Desventajas:**
 - **Performance:** Requiere múltiples viajes de ida y vuelta al servidor (o *client-side joins*), lo que ralentiza la recuperación de datos complejos.
 - **Sin Integridad Referencial:** MongoDB no gestiona claves foráneas; la responsabilidad de asegurar que un ID referenciado exista recae totalmente en la **lógica de la aplicación**.

Comparación con Bases de Datos Relacionales (RDBMS)

Característica	RDBMS Tradicional	MongoDB (Agregados)
Estructura	Datos divididos en tablas normalizadas.	Datos agrupados en estructuras ricas (Agregados).
Vínculos	Claves foráneas gestionadas por el motor.	Incrustación (dentro del doc) o Referencias (manuales).
Resolución	Se calculan uniones (JOIN) al momento de la consulta.	Se favorece tener los datos ya "unidos" (embebidos) para acceso rápido.
Atomicidad	Transacciones ACID que pueden abarcar múltiples tablas.	Transacciones atómicas limitadas a un solo documento .
Flexibilidad	Esquema rígido y fijo definido de antemano.	"Sin esquema" (**schemaless**); cada

Conclusión: Mientras que el RDBMS prioriza la consistencia y la falta de redundancia mediante la normalización, MongoDB prioriza el **rendimiento de acceso y la escalabilidad horizontal** mediante el uso de agregados embebidos, dejando las referencias solo para casos donde los datos no encajan en una unidad lógica o superan límites de tamaño.

Inciso 8

Tomando como referencia el modelo de los trabajos practicos anteriores y suponiendo que este podria mapearse a una base de datos en MongoDB, proponer algunos casos donde la relacion seria conveniente mapearla como referencia y otros como documentos embebidos. Justificar la eleccion.

Respuesta:

Propuesta de Diseño en MongoDB

Embebidos (Documentos anidados)

Relación	Justificación
Route → Stops	Las paradas siempre se consultan junto con la ruta, son datos acotados (cantidad finita) y típicamente no se reutilizan independientemente en otras consultas
Purchase → ItemService (lista)	Los servicios contratados son parte intrínseca de la compra; no se comparten con otras compras ni se accede a ellos por separado
Route → DriverList / TourGuideList	Listas pequeñas de asignación de personal; siempre asociadas a esa ruta específica

Referencias (DBRef / ObjectId)

Relación	Justificación
<code>Service</code> → <code>Supplier</code>	Un proveedor puede ofrecer muchos servicios. Se accede al proveedor independientemente de sus servicios (ej: lista de proveedores)
<code>Service</code> → <code>ItemService</code> (relación inversa)	Los items se consultan frecuentemente desde el Service para ver en qué compras fueron incluidos
<code>Purchase</code> → <code>User</code>	El usuario es una entidad independiente con su propia vida; puede tener múltiples compras asociadas
<code>Purchase</code> → <code>Route</code>	Las rutas son recursos reutilizables en múltiples compras; se accede a la ruta independientemente
<code>Purchase</code> → <code>Review</code>	Las reviews pueden consultarse independientemente (ej: todas las reviews de un usuario, o reviews por puntuación)
<code>Route</code> ↔ <code>Stop</code> (relación N:N)	Las paradas son entidades que pueden aparecer en múltiples rutas; conviene modelar como referencia

Justificación General

- **Embebidos:** Se utilizan cuando el objeto secundario no tiene sentido independiente o se consulta siempre junto con su documento padre.
- **Referencias:** Se utilizan cuando existen relaciones muchos-a-muchos o los objetos se comparten entre diferentes documentos. También cuando se requiere acceder a la entidad independientemente con frecuencia.

II. Operaciones CRUD basicas

Descargar la ultima version de MongoDB desde el sitio oficial e ingresar al cliente de linea de comando para realizar los siguientes ejercicios.

1. Configuracion inicial e inserciones

Inciso 9

Crear una nueva base de datos llamada "tours" y una coleccion llamada "recorridos"

Respuesta

Luego de instalar MongoDB

```
mongosh # Paso 1

use tours # Paso 2

db.createCollection("recorridos") # Paso 3

show collections # Paso 4 (opcional)
```

Inciso 10

En la nueva coleccion, utilizando el comando correspondiente, insertar el siguiente documento:

```
{ "nombre": "City Tour", "precio": 200, "stops": ["Diagonal Norte", "Avenida de Mayo", "Plaza del Congreso"], "totalKm": 5 }
```

Respuesta:

```
db.recorridos.insertOne({ "nombre": "City Tour", "precio": 200, "stops": ["Diagonal Norte", "Avenida de Mayo", "Plaza del Congreso"], "totalKm": 5 })

db.recorridos.find() # Para convalidar resultado
```

Inciso 11

Recuperar la informacion insertada usando db.recorridos.find() (puede agregarse .pretty() al final para ver los datos indentados). ¿Que diferencia se observa entre el documento insertado y el documento recuperado?

Respuesta:

Al recuperar un documento en MongoDB mediante el comando `db.recorridos.find()`, se observa una diferencia fundamental respecto al

documento originalmente insertado: la presencia del campo `_id`.

Todo documento almacenado en una colección de MongoDB requiere un campo llamado `_id`, el cual actúa como su **clave primaria única**. Si el desarrollador no define explícitamente este campo durante la operación de inserción, el motor de la base de datos lo genera automáticamente antes de persistir el registro en el disco.

Inciso 12

Agregar a la colección, utilizando un solo comando, los documentos especificados en el archivo "material_adicional_1.json" adjunto a esta práctica.

```
db.recorridos.insertMany([
  {
    "nombre": "Historical Adventure",
    "precio": 30000,
    "stops": ["Avenida de Mayo", "Plaza del Congreso", "Río de la Plata", "Teatro Colón", "Av 9 de Julio", "Plaza Italia"],
    "totalKm": 10
  },
  {
    "nombre": "Architectural Expedition",
    "precio": 50000,
    "stops": ["Usina del Arte", "Puerto Madero", "Museo Nacional de Bellas Artes", "Museo Moderno", "Museo de Arte Latinoamericano", "Teatro Colón", "San Telmo", "Recoleta"],
    "totalKm": 12
  },
  {
    "nombre": "Delta Tour",
    "precio": 80000,
    "stops": ["Río de la Plata", "Bosques de Palermo", "Delta"],
    "totalKm": 8
  },
  {
    "nombre": "Nature Escape",
    "precio": 40000,
    "stops": ["Delta", "Río de la Plata", "Av 9 de Julio", "Puerto Madero"]
  }
])
```

```
},
{
  "nombre": "Artistic Journey",
  "precio": 60000,
  "stops": ["Museo Nacional de Bellas Artes", "Teatro Colón", "Usina del Arte", "Planetario", "San Telmo", "La Boca - Caminito", "Belgrano - Barrio Chino", "Av 9 de Julio"],
  "totalKm": 15
},
{
  "nombre": "Tango Experience",
  "precio": 35000,
  "stops": ["Avenida de Mayo", "Plaza del Congreso", "Paseo de la Historieta", "San Telmo", "La Boca - Caminito", "Recoleta"],
  "totalKm": 10
},
{
  "nombre": "Gastronomic Delight",
  "precio": 70000,
  "stops": ["San Telmo", "La Boca - Caminito", "Belgrano - Barrio Chino", "Recoleta", "Costanera Sur"],
  "totalKm": 9
},
{
  "nombre": "Urban Exploration",
  "precio": 45000,
  "stops": ["Avenida de Mayo", "Museo Moderno", "Paseo de la Historieta", "Usina del Arte", "San Telmo", "La Boca - Caminito", "Belgrano - Barrio Chino", "Recoleta", "El Monumental (Estadio River Plate)", "Costanera Sur", "Plaza Italia"],
  "totalKm": 14
},
{
  "nombre": "Museum Tour",
  "precio": 55000,
  "stops": ["Museo Nacional de Bellas Artes", "Teatro Colón", "Planetario", "Bosques de Palermo", "San Telmo", "La Boca - Caminito", "Recoleta", "El Monumental (Estadio River Plate)", "Av 9 de Julio"],
  "totalKm": 13
}
```

```
},
{
  "nombre": "Historic Landmarks",
  "precio": 25000,
  "stops": ["Avenida de Mayo", "Plaza del Congreso", "Usina
del Arte", "San Telmo", "La Boca - Caminito", "Belgrano -
Barrio Chino", "Recoleta", "El Monumental (Estadio River
Plate)", "Costanera Sur", "Plaza Italia"],
  "totalKm": 23
},
{
  "nombre": "Temporal Route",
  "precio": 50000,
  "stops": ["Av 9 de Julio", "Avenida de Mayo",
"Planetario"],
  "totalKm": 5
},
{
  "nombre": "City Lights",
  "precio": 50000,
  "stops": ["Avenida de Mayo", "Paseo de la Historieta",
"Usina del Arte", "Recoleta", "El Monumental (Estadio River
Plate)", "Costanera Sur", "Plaza Italia", "Museo de Arte
Latinoamericano"],
  "totalKm": 10
},
{
  "nombre": "Cultural Odyssey",
  "precio": 65000,
  "stops": ["Bosques de Palermo", "San Telmo", "La Boca -
Caminito", "Recoleta", "El Monumental (Estadio River Plate)",
"Av 9 de Julio", "Plaza Italia"],
  "totalKm": 11
},
{
  "nombre": "Riverfront Ramble",
  "precio": 35000,
  "stops": ["Río de la Plata", "Bosques de Palermo", "El
Monumental (Estadio River Plate)", "Costanera Sur"],
  "totalKm": 6
}
```

```
}  
1)
```

2. Operaciones de actualizacion y eliminacion

Inciso 13

Actualizar el recorrido "Cultural Odyssey" para que su total de kilometros sea 12.

Respuesta:

```
db.recorridos.updateOne(  
  { nombre: "Cultural Odyssey" }, // El filtro para encontrar  
  el registro  
  { $set: { totalKm: 12 } }      // La actualización  
  propiamente dicha  
)
```

Inciso 14

Actualizar el listado de stops del recorrido "Delta Tour" para agregar "Tigre".

Respuesta

```
db.recorridos.updateOne({ nombre: "Delta Tour" }, { $push: {  
  stops: "Tigre" } })
```

Inciso 15

Aumentar un 10% el precio de todos los recorridos

Respuesta

```
db.recorridos.updateMany( { }, // Filtro vacío: aplica a todos  
  los documentos  
  { $mul: { precio: 1.1 } }    // Multiplica el campo precio  
  por 1.1  
)
```

Inciso 16

Eliminar el recorrido con nombre "Temporal Route".

Respuesta

```
db.recorridos.deleteOne({ nombre: "Temporal Route" }) #  
Eliminación física
```

Inciso 17

Crear el array de etiquetas (tags) para la ruta "Urban Exploration" y agregar el elemento "Gastronomia" a dicho arreglo.

Respuesta

```
db.recorridos.updateOne(  
  { nombre: "Urban Exploration" },  
  { $set: { tags: ["Gastronomia"] } }  
)
```

3. Consultas con find()

Inciso 18

Obtener la ruta con nombre "Museum Tour".

Respuesta

```
db.recorridos.find({nombre: "Museum Tour"})
```

Inciso 19

Las rutas con precio superior a \$60.000.

Respuesta

```
db.recorridos.find({precio: {$gt: 60000}})
```

Inciso 20

Las rutas con precio superior a \$50.000 y con un total de kilometros mayor a 10.

Respuesta

```
db.recorridos.find({precio: { $gt: 50000 }, totalKm: { $gt: 10 } })
```

Inciso 21

Las rutas que incluyan el stop "San Telmo".

```
db.recorridos.find({ stops: "San Telmo" })
```

Inciso 22

Las rutas que incluyan el stop "Recoleta" y no el stop "Plaza Italia".

```
db.recorridos.find({
  stops: { $eq: "Recoleta", $ne: "Plaza Italia" }
})
```

Inciso 23

El nombre y el total de km (si es que posee) de las rutas que incluyan el stop "Delta" y tengan un precio menor a \$50.000.

```
db.recorridos.find(
  { stops: "Delta", precio: { $lt: 50000 } },
  { nombre: 1, totalKm: 1, _id: 0 }
)
```

Inciso 24

Las rutas que incluyen tanto "San Telmo" como "Recoleta" y "Avenida de Mayo" entre sus stops.

```
db.recorridos.find({
  stops: { $all: ["San Telmo", "Recoleta", "Avenida de Mayo"]
}
})
```

Inciso 25

Solo el nombre de las rutas que dispongan de mas de 5 stops.

```
db.recorridos.find(
  { $expr: { $gt: [ { $size: "$stops" }, 5 ] } },
  { nombre: 1, _id: 0 }
)
```

Inciso 26

Las rutas que no tengan definido el total de sus kilometros.

```
db.recorridos.find({totalKm: {$exists: false}})
```

Inciso 27

Los nombres y el listado de stops de aquellas rutas que incluyen algun museo en sus recorridos.

```
db.recorridos.find(
  { stops: { $regex: "Museo", $options: "i" } },
  { nombre: 1, stops: 1, _id: 0 }
)
```

- La "i" significa_case-insensitive.

Inciso 28

La cantidad total de elementos que posee la coleccion.

```
db.recorridos.find().count()
```

III. Aggregation Framework

Aggregation Framework es una herramienta de MongoDB que permite ampliar la posibilidad de uso de su lenguaje de consulta. Posibilita realizar consultas complejas, filtrado, agrupacion y analisis de datos en tiempo real,

aplicando una serie de etapas de procesamiento secuencialmente sobre los documentos de una coleccion.

DOCUMENTACIÓN AGGREGATE

1. Configuracion

Inciso 29

Crear una nueva base de datos llamada "tours2". Guardar el archivo "generador1.js" adjunto a esta practica y ejecutarlo con:

```
load(<ruta del archivo 'generador1.js'>)
```

Si se utiliza un cliente que lo permita (por ejemplo Robo3T o MongoDB Compass), se puede ejecutar directamente en el espacio de consultas. Examinar las colecciones generadas antes de continuar.

2. Consultas con Aggregation Framework

Inciso 30

Obtener una muestra de 5 rutas aleatorias de la coleccion.

```
db.route.aggregate([ { $sample: { size: 5 } } ])
```

- **aggregate([])** : Es el método que permite pasar una lista de pasos (stages) para procesar los datos.
- **\$sample** : Es la etapa específica para selección aleatoria.
- **size: 5** : Indica la cantidad de documentos que querés obtener.

Inciso 31

Extender la consulta anterior para incluir en el resultado toda la informacion de cada una de las stops. Tener en cuenta que pueden ligarse por su codigo.

```
db.route.aggregate([  
  { $sample: { size: 5 } },  
  {  
    $lookup: {  
      from: "stop",
```

```

        localField: "stops",
        foreignField: "code",
        as: "detalle_paradas"
    }
}
])

```

\$lookup

Es una etapa del pipeline de agregación que realiza un **left outer join** entre dos colecciones. Esto significa que trae todos los documentos de la colección "A" y les pega la información coincidente de la colección "B".

- **Identificación del Origen** (**from**): Se especifica la colección externa de donde queremos sacar los datos adicionales.
- **Referencia Local** (**localField**): Se elige el campo del documento actual (el que estamos procesando) que contiene el valor de referencia (la "FK").
 - *Nota:* Si este campo es un **arreglo**, Mongo buscará cada valor del arreglo automáticamente.
- **Referencia Externa** (**foreignField**): Se indica cuál es el campo en la otra colección que debe coincidir con nuestro valor local (casi siempre es el **_id**).
- **Generación del Resultado** (**as**): Se define el nombre del **nuevo campo** que se creará en nuestro documento.

Inciso 32

Obtener la información de las rutas (incluyendo la de sus stops) que tengan un precio mayor o igual a \$90.000.

```

db.route.aggregate([
  { $sample: { size: 5 } },
  { $match: {
    price: { $gt: 90000 }
  } },
  {
    $lookup: {
      from: "stop",
      localField: "stops",

```

```

        foreignField: "code",
        as: "detalle_paradas"
    }
}
])

```

Inciso 33

Obtener la informacion de las rutas que tengan 5 stops o mas.

```

db.route.aggregate([
  {
    $match: {
      $expr: { $gte: [ { $size: "$stops" }, 5 ] }
    }
  },
  {
    $project: { name: 1, stops: 1, _id: 0 }
  }
])

```

Inciso 34

Obtener la informacion de las rutas que tengan incluido en su nombre el string "111".

```

db.route.aggregate([
  {
    $match: {
      name: { $regex: "111" }
    }
  }
])

```

Inciso 35

Obtener solo las stops de la ruta con nombre "Route100".

```

db.route.aggregate([
  {

```

```

    $match: {
      name: { $regex: "^Route100$", $options: "i" }
    },
    {
      $lookup: {
        from: "stop",
        localField: "stops",
        foreignField: "code",
        as: "informacion_paradas"
      }
    },
    {
      $project: {
        _id: 0,
        informacion_paradas: 1
      }
    }
  ]
})

```

- **\$match** : Filtramos para que solo pase la ruta que se llama exactamente "Route100". El uso de `^` y `$` en el regex asegura que el nombre sea igual a eso y no que simplemente lo contenga (como "Route1000").
- **\$lookup** : "Cruza" los números que tenés en el arreglo `stops` de la ruta con el campo `code` de la colección `stop`.
- **\$project** : Este paso es opcional pero útil para lo que pedís ("obtener solo las stops"). Al poner `informacion_paradas: 1`, le decís a MongoDB que solo te devuelva ese campo, ocultando el precio, los kms y el ID de la ruta.

Inciso 36

Obtener la informacion del stop que mas apariciones tiene en rutas.

```

db.route.aggregate([
  { $match: { stops: { $exists: true } } },
  { $unwind: "$stops" },
  { $group: {
    _id: "$stops",
    total: { $sum: 1 }
  } }
])

```

```

    }},

    { $sort: { total: -1 } },
    { $limit: 1 },

    { $lookup: {
      from: "stop",
      localField: "_id",
      foreignField: "code",
      as: "info"
    }},
  ]),

```

1. **\$match (Limpieza)**: Filtra rutas sin paradas (`stops.0: { $exists: true }`). Evita procesar documentos vacíos o nulos, optimizando el inicio de la cadena.
2. **\$unwind (Fragmentación)**: Descompone cada array de `stops` . Si una ruta tiene 5 paradas, genera 5 documentos individuales. Es el paso obligatorio para poder contar elementos internos de un arreglo.
3. **\$group (Cálculo)**: Agrupa por el ID/Código de la parada y usa el acumulador `$sum: 1` . Transforma los miles de documentos individuales en una lista única de paradas con su respectivo contador.
4. **\$sort + \$limit (Podio)**: Ordena por el contador de forma descendente (`-1`) y toma solo el primer registro. Esto aísla al "ganador" de la lista de forma eficiente.
5. **\$lookup (Enriquecimiento)**: Realiza un *left outer join* con la colección `stop` . Cruza el código obtenido con la información real (nombre, descripción) para que el resultado sea legible y no solo un número.

Inciso 37

Obtener las rutas con precio inferior a \$15.000. Agregar a cada una una nueva propiedad que especifique la cantidad de stops que posee. Crear una nueva colección llamada "rutas_economicas" y almacenar estos elementos.

```

db.route.aggregate([
  {
    $match: { price: { $lt: 15000 } }
  },
  {

```



```
    $addFields: { cantStops: { $size: "$stops" } }  
  },  
  {  
    $out: "rutas_economicas"  
  }  
])
```

Inciso 38

Por cada stop existente en la coleccion, calcular el precio promedio de las rutas que la incluyen